1. Largest Difference in List
Problem:
Given a list of integers, return the largest difference between any two elements, where the larger element appears after the smaller one. If no such pair exists, return 0.

Example:
Input: [1, 2, 90, 10, 110]
Output: 109
Explanation: The largest difference is between 1 and 110.

2. First Non-Repeating Character

Problem:
Given a string, return the first non-repeating character. If there are no non-repeating characters, return None.

Example:
Input: "aabcc"
Output: 'b'
Explanation: The first non-repeating character is 'b'.

3. Check If Strings Are Permutations

Problem:
Given two strings, write a function to check if one is a permutation of the other. Return True if they are permutations, False otherwise.

Example:
Input: "abc", "bca"
Output: True
Explanation: Both strings contain the same characters in any order.

4. Find the Second Largest Unique Number

Problem:
Given a list of integers, return the second largest unique number. If no second largest unique number exists, return None.

Example:
Input: [4, 5, 6, 6, 7]
Output: 6
Explanation: The second largest unique number is 6.

5. Count Distinct Pairs

Problem:
Given a list of integers and a target sum, count how many distinct pairs of integers in the list sum up to the target.

Example:
Input: [1, 2, 3, 4, 3, 6], Target: 6
Output: 2
Explanation: The pairs (2, 4) and (1, 5) sum up to 6.

6. Remove Duplicates from List

Problem:
Given a list of integers, remove all duplicates while maintaining the order of their first Occurrence.

Example:
Input: [1, 2, 2, 3, 4, 5, 1]
Output: [1, 2, 3, 4, 5]
Explanation: Duplicates are removed, but the order of elements is preserved.

7. Subsets of a List

Problem:
Given a list of integers, return all possible subsets (the power set) of the list.
Example:
Input: [1, 2, 3]
Output: [[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]
Explanation: The function returns all subsets of the list.

8. Longest Increasing Subsequence

Problem:
Given a list of integers, find the length of the longest subsequence of strictly increasing Elements.

Example:
Input: [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4
Explanation: The longest increasing subsequence is [2, 3, 7, 101].

9. Maximum Product of Two Numbers

Problem:
Given a list of integers, return the maximum product of any two numbers in the list.

Example:
Input: [1, 10, -5, 1, 100]
Output: 1000
Explanation: The largest product comes from 10 * 100 = 1000.

10. Frequency of Characters in String

Problem:
Given a string, return the character that appears most frequently. If there is a tie, return the lexicographically smallest character.
Example:
Input: "aabbbc"
Output: 'b'
Explanation: The most frequent character is 'b'.

11. Print Prime Numbers Less Than N

Problem:
Write a function that prints all prime numbers less than a given number n.
Example:
Input: 10
Output: 2, 3, 5, 7

12. Sum of Variable-Length Arguments

Problem:
Write a function that takes a variable number of arguments (*args) and returns the sum of all the numbers.
Example:
Input: (2, 3, 4)
Output: 9
Explanation: The sum of 2 + 3 + 4 is 9.

13. Find the Student with Highest Score
Problem:
Given a dictionary of students' names and their scores, write a function that returns the name of the student with the highest score.

Example:
Input: {"Alice": 85, "Bob": 90, "Charlie": 92}
Output: "Charlie"

14. Apply Lambda to List
Problem:
Write a function that takes a list of integers and a lambda function, and returns a new list where the lambda function is applied to each element of the list.

Example:
Input: [1, 2, 3, 4], Lambda: lambda x: x ** 2
Output: [1, 4, 9, 16]
Explanation: Each number is squared by the lambda function.

15. Sum or Multiply Based on Keyword Argument
Problem:
Write a function that takes a variable-length list of numbers (*args) and an optional keyword argument operation. If operation='multiply', return the product of all the numbers; otherwise, return their sum.

Example:
Input: (1, 2, 3, 4), operation='multiply'
Output: 24
Explanation: The product of the numbers 1 * 2 * 3 * 4 is 24.

Instructions for the Group Activity:
● Group Size: Divide class into small groups (3-4 students per group).
● Duration: 55 mins
● Tools: Each group should have access to a computer with a Python interpreter(Colab or VS Code).
● Deliverable: Each group must submit their GitHub link, with clear comments, explanations for each part, and any outputs or results from the tasks.
● Objective: This activity will test your ability to work with data structures, control flow (loops and conditionals), functions, and user input. It also helps you practice collaborative problem-solving.

Grading Criteria for Individual Questions:
● Correctness: Does the code return the correct outputs for the given inputs?(70%)
● Code Structure: Is the code clean, well-organized, and commented?(15%)
● Functionality: Are all tasks completed and tested?(5%)
● Collaboration: Did the group work well together and discuss solutions?(5%)